# StockScan - Design Document

AI-assisted stock analysis platform: architecture, key design concepts and major functions

Versio

### Contents

1. Pur

## 1. Purpose and scope

StockScan is a web application for analyzing listed stocks with a set of AI 'analyst agents', each of which applies a named investor's framework (Warren Buffett, Jamie Dimon, John Murphy) to data that has been gathered and stored in advance. It supports single analyses, weekly point-in-time backtests over a year, and a funnel scan across the S&P 500. Every analysis is stored with its inputs, outputs and reports so that a decision can be audited and reproduced.

This document describes the key design concepts, the architecture, the major functions and the principal data stores. Diagnostics and operational detail are out of scope; only the major database tables are covered.

*All agent output is model-generated analysis for a reader who makes their own decisions. It is not investment advice, and the application says so on every result page and report.*

## 2. Key design concepts

### 2.1 Skills as files, not code

Each analyst agent is a skill folder in the repository: skills/<id>/SKILL.md holds the investor's framework, decision rules and evidence rules as markdown with a small frontmatter (name, description, version, data_source); skills/<id>/data-structure.json is the data contract - a JSON Schema for exactly what the agent receives (input) and exactly what it must return (output). The markdown body becomes the model's system prompt; the output schema is enforced with structured outputs. Changing how an agent reasons, or adding a new agent, is a file change, not a code change.

- The input schema also drives data gathering: fundamentals-based skills list the quote-summary fields and statement rows they need, and the backend fetches exactly those.

- Three rules are enforced by the server regardless of skill: a BUY must carry a target price and a stop price; a SELL must carry a target price; inconsistent prices (e.g. a BUY target below the current price) are flagged as warnings.

### 2.2 Point-in-time discipline

A backtest or scan run 'as of' a date must see only what existed on that date. The system enforces this in the data, not in the prompt alone:

- Prices: only closes strictly before (backtest) or on/before (scan, technical) the analysis date.

- Financial reports: annual and quarterly statements are visible only once reported - using the company's actual earnings-release dates where known, otherwise a 90-day (annual) / 45-day (quarterly) lag. Ratios such as P/E, ROE and margins are rebuilt from the statements visible at the time, because the data vendor's live ratios cannot be rewound.

- Technical data: swing pivots carry the date they became known (pivot date + confirmation window); support/resistance levels are stored per day from pivots confirmed by that day; chart patterns carry their neckline-break date.

- The model is told that the analysis date is 'today' and that nothing after it exists. One limitation remains and is disclosed everywhere: the model's own training knowledge cannot be rewound, so results test the framework on the data, not a hindsight-free analyst.

### 2.3 Precompute once, analyze offline

Model runs never fetch from the internet or compute indicators. Everything an agent needs is prepared beforehand:

- A per-symbol data bundle (profile, 49 quote-summary fields, every annual and quarterly statement line item, earnings dates, two years of prices) is downloaded once and kept in Azure SQL and in Vercel Blob as browsable files.
- For technical analysis, nine TECH_ tables are computed from stored prices: daily and weekly bars, 38 indicators, trend classification, confirmed pivots, per-day support/resistance and Fibonacci levels, signal events and detected patterns.
- Loading is incremental: prices are fetched only for days after the last stored close, and technical data is recomputed only when it is older than the latest close. Re-running a load over the whole index is cheap.

### 2.4 Resumable, batched jobs

The backend runs as serverless functions with a per-request time limit, and a model call takes 30-120 seconds. Long work is therefore modelled as a job with one unit of work per request: the page creates the job (rows in a table), then calls a /step endpoint in a loop; each step claims the next pending unit atomically, runs exactly one model call (or one symbol load), and persists the result. Jobs can be stopped, resumed and retried, several browser workers can drive the same job in parallel, and the page polls the database so progress is live even when another tab is doing the work.

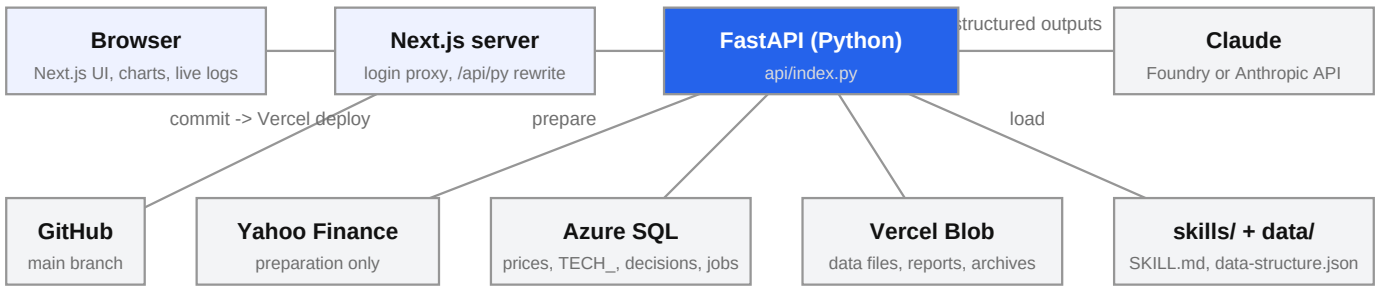
### 2.5 Evidence and provenance

- Every decision stores the exact JSON the model saw, the JSON it returned, the list of reports and data sets consumed (each statement by period and publication date, the price range, the profile), token usage and a timestamped run log.
- Agents must cite the source table and field for every metric, level, reading and pattern, and list every judgment call under 'assumptions' and every missing input under 'data gaps'.
- Two reports are generated per decision: an interactive HTML report (for technical skills: a candlestick chart with indicators, levels, signals, patterns and the verdict lines, and an evidence panel that jumps the chart to each cited item) and a PDF. Both are stored in Vercel Blob next to the run's input/output JSON.

### 2.6 Cost control: cheapest evidence first

The S&P 500 scan orders work by cost. A free technical pre-screen from the TECH_ tables (primary trend up, intermediate trend not down, optional minimum ADX) decides which (date, stock) pairs reach a model at all. John Murphy runs first; only a BUY with the close inside the entry zone proceeds to Warren Buffett; only a Buffett BUY proceeds to Jamie Dimon. A rejection at any stage settles the pair immediately, so no further model calls are spent on it.

## 3. Architecture




| Component | Role |
|---|---|
| GitHub + Vercel | Source control and hosting. The Vercel project is linked to the GitHub repository; every commit to main is built and deployed automatically (see section 8). |
| Next.js 16 (React, TypeScript) | User interface: pages, calendars, live logs, charts (lightweight-charts). A proxy enforces the single-user login and forwards /api/py/* to the Python API. |

| Component | Role |
|---|---|
| FastAPI (Python, api/index.py) | All business logic: data gathering, point-in-time payloads, technical computation, skill loading, model calls, validation, reports, job runners. |
| Azure SQL | System of record: prices, data bundles, TECH_ tables, constituents, backtests and decisions, scan and load jobs. |
| Vercel Blob | Document store: per-symbol data files, per-decision HTML/PDF reports and input/output JSON archives. |
| Claude (Microsoft Foundry or Anthropic API) | The analyst model. Same SDK either way; provider chosen by environment variables. Structured outputs enforce each skill's output schema. |
| Yahoo Finance (yfinance) | Source of prices, quote summary, financial statements and earnings dates - used only during preparation, never during a run. |

Authentication is a single configured user: a signed, HttpOnly session cookie issued by Next.js and verified both by the Next.js proxy and by the Python API, so the API cannot be reached around the front end. All secrets come from environment variables; nothing is stored in code or the database.

# 4. Major functions

## 4.1 Load Prices and the S&P 500 universe

Load Prices stores two years of daily prices for a symbol. The S&P 500 universe page processes every constituent in dbo.SP500_CONSTITUENTS as a batched job: delta price updates, then technical-data recomputation only where stale; every stock is shown as a tile coloured by state (missing / stale / ready) and, during a job, by progress.

## 4.2 Single stock analysis (charts)

An interactive multi-pane chart from the stored prices: candlesticks with moving averages (5/10/20/30/50/200) and crossovers, Bollinger Bands, support/resistance from pivots, volume with confirmation markers, RSI, MACD and relative strength versus chosen index and sector benchmarks, with signal tiles and a table view.

## 4.3 AI agent: skill library and single runs

The skill library lists every skill with its SKILL.md and data-structure.json. A single run applies a skill to a symbol as of today and shows the verdict, prices, checklist, sections, metrics with sources, risks, assumptions and data gaps.

## 4.4 Weekly backtest

Runs a skill as of the first N Mondays of a year (default 10, up to 52). A calendar shows every day grey, scheduled Mondays outlined, the running one pulsing, completed ones green with the verdict letter (B/H/S). Clicking a completed day opens the decision page: run date, key conclusion, references, the embedded interactive HTML report and the PDF. The page shows the configuration in use and a live, timestamped run log for the Monday in progress (data assembled, model connected, tokens, first output, progress, verdict, reports stored).

## 4.5 S&P 500 scan

For each Monday (or weekday) in a date range and each constituent, runs the funnel described in 2.6. The page shows the exact number of stage-1 model calls before anything is spent, a progress bar per date (stocks analyzed vs total), funnel counters, a calendar whose dates carry the number of stocks recommended by all three agents, a per-date table with each agent's verdict linked to its report, an embedded point-in-time technical chart for any stock, and the full hit list.

## 4.6 Run history

Every run with its agent, timing, date range, counts, verdict tallies, overall status and the exact reasons for any failures, expandable to the per-Monday detail.

# 5. The analyst agents

| Agent | Framework (from the skill file) | Data it uses | Distinctive outputs |
|---|---|---|---|
| Warren Buffett | Circle of competence, durable moat, management quality, financial strength, owner-earnings intrinsic value, margin of safety (>= 25%). | Fundamentals bundle: quote summary and annual statements (point-in-time for backtests). | Intrinsic value low/base/high, buy-below price, valuation inputs (growth, discount rate, terminal growth). |

| Agent | Framework (from the skill file) | Data it uses | Distinctive outputs |
|---|---|---|---|
| Jamie Dimon | Fortress balance sheet, risk discipline, through-the-cycle earnings, management and culture, competitive position, macro/regulatory exposure; mandatory 3-scenario stress test. | Fundamentals bundle plus liquidity, leverage, coverage, governance-risk fields and debt flows. | Balance-sheet grade, stress test (base / recession / severe), normalized-earnings valuation. |
| John Murphy | Technical Analysis of the Financial Markets: Dow Theory, trend and support/resistance, moving averages, oscillators and divergences, volume confirmation, patterns with measured moves, Fibonacci, trading rules. | TECH_ tables only: bars, indicators, trend, confirmed pivots, levels, signals, patterns as of the date. | Trend assessment, key levels, indicator readings, patterns, trade plan (entry zone, stop basis, target method, reward-to-risk). |

All three return the same core verdict: recommendation (BUY / HOLD / SELL), conviction, headline, current price, buy-below price, target price, stop price with rationales, time horizon, a summary memo, a checklist, supporting sections, key metrics with sources, risks, assumptions and data gaps.

# 6. Principal data stores

| Table / store | Purpose |
|---|---|
| dbo.stock_prices | Daily OHLCV per symbol; the base for charts, price snapshots and technical computation. |
| dbo.symbol_data (+ Blob data/<SYMBOL>/) | Per-symbol data bundle: profile, quote summary, all statement line items, earnings dates, prices; dated snapshots in Blob. |
| TECH_PRICE_DAILY / WEEKLY, TECH_INDICATOR_DAILY, TECH_TREND_DAILY, TECH_PIVOT, TECH_LEVEL, TECH_SIGNAL, TECH_PATTERN, TECH_LOAD | Precomputed technical analysis per symbol and trading day, with confirmation dates for point-in-time reads. |
| dbo.SP500_CONSTITUENTS | The index membership used by the universe loader and the scan; maintained in the database (seeded from the repository list). |
| dbo.agent_backtests, dbo.agent_decisions | One row per run and one per analysis (date, symbol, skill): verdict fields, input/output JSON, references, warnings, tokens, log, report paths. |
| dbo.SCAN_JOB, dbo.SCAN_ITEM | S&P 500 scan jobs and the state of every (date, stock) pair through the funnel. |
| dbo.UNIVERSE_LOAD_JOB / _ITEM | Batched index loading jobs and per-symbol outcomes. |
| Vercel Blob reports/ | Per-decision PDF, HTML report, and the model's input and output JSON. |

# 7. Reports

The HTML report is a self-contained page: verdict card with target/stop/buy-below, an evidence panel of every cited level, reading, pattern and metric with its source, the narrative and structured blocks, references and, for technical skills, the interactive chart described in 2.5. The PDF carries the key conclusion first, then the full analysis, then the data and reports referenced. Both are produced when a decision completes and regenerated on demand if the stored copy is missing.

# 8. Deployment and configuration

## 8.1 Continuous deployment from GitHub

The source of truth is the GitHub repository (main branch). The Vercel project is linked to it, so every commit pushed to main triggers a Vercel build and deployment automatically: Vercel builds the Next.js application, packages api/index.py as a Python serverless function (with skills/ and data/ bundled alongside it, per vercel.json) and switches production to the new deployment when the build succeeds. A failed build leaves the previous deployment live. Preview deployments are created for other branches and pull requests. No manual deploy step exists; releasing is committing.

- Code, skills (SKILL.md, data-structure.json) and the S&P 500 seed list therefore go live together with the commit that changes them.
- Database tables are created and migrated automatically by the Python API on first use after a deployment, so schema changes need no separate release step.
- Rolling back is redeploying an earlier commit from the Vercel dashboard, or reverting the commit on main.

## 8.2 Runtime and configuration

- Hosted on Vercel: the Next.js app plus one Python serverless function (api/index.py) with a 300-second limit (Fluid Compute); long work is split into per-request steps as described in 2.4.

- Configuration is entirely environment variables set in the Vercel project (and in a local .env.local for development): the model provider (Microsoft Foundry resource and key, or an Anthropic API key), the model and reasoning effort (default medium), the Azure SQL login, the Vercel Blob token and the single login credentials. A change to a variable takes effect on the next deployment.

- Secrets are never stored in code, the repository or the database; the Diagnosis pages show only whether each variable is set.

## 9. Known limitations and next steps

- The model's training knowledge cannot be rewound to a past date; backtests test the framework on the data, not a hindsight-free analyst.

- Market data comes from Yahoo Finance via yfinance, whose terms limit the data to personal, non-commercial use.

- Pattern detection covers double tops/bottoms and head-and-shoulders; continuation patterns and trendlines are left for the agent to infer and flag as assumptions. No intermarket or sector-relative data is supplied to the technical agent.

- A full S&P 500 scan is large (hundreds to thousands of model calls per date range even after pre-screening); runs are designed to be stopped, resumed and parallelized, and the page states the call count before starting.

- Possible next steps: scheduled nightly data refresh, a scan outcome tracker (what happened after each hit), portfolio-level views, and additional analyst skills.